Tugas 2 - Membuat Sistem Informasi Manajemen Data Sederhana

Kasus : Apotek (FEFO)

Nama: Muhammad Rifky Sandi Yudha
NIM : 10124027
Kelas : IF-1

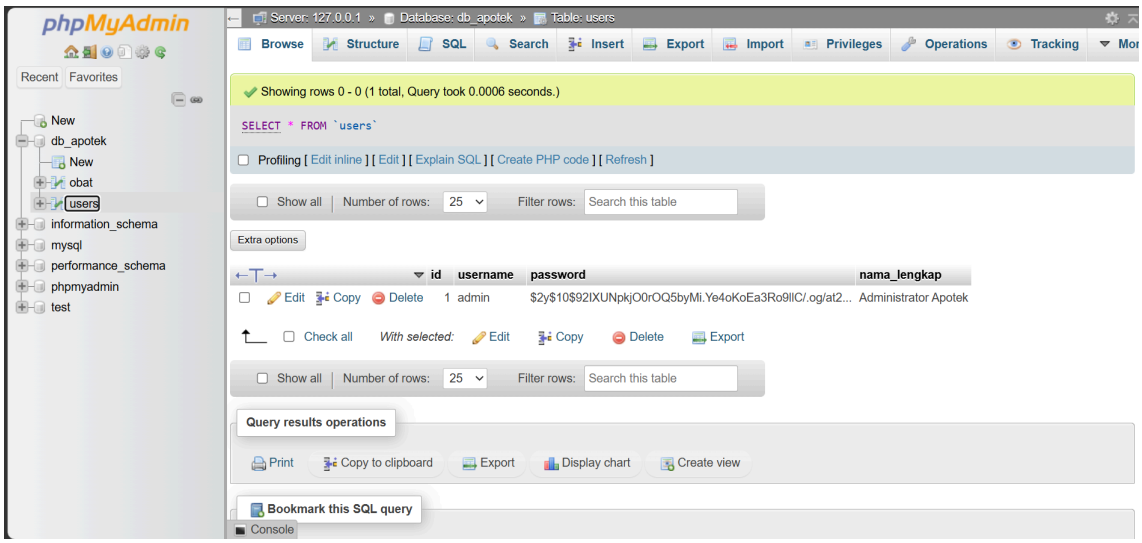
Username dan Password untuk masuk ke dashboard :
Username : admin
Password : password

1. Skema Database (MySQL)

Sebelum menulis kode PHP, langkah pertama yang saya lakukan adalah merancang dan membuat database di MySQL (melalui phpMyAdmin) dengan nama *db_apotek*. Di dalam database ini, saya membuat dua tabel utama agar sistem bisa berjalan sesuai ketentuan tugas:

A. Tabel users

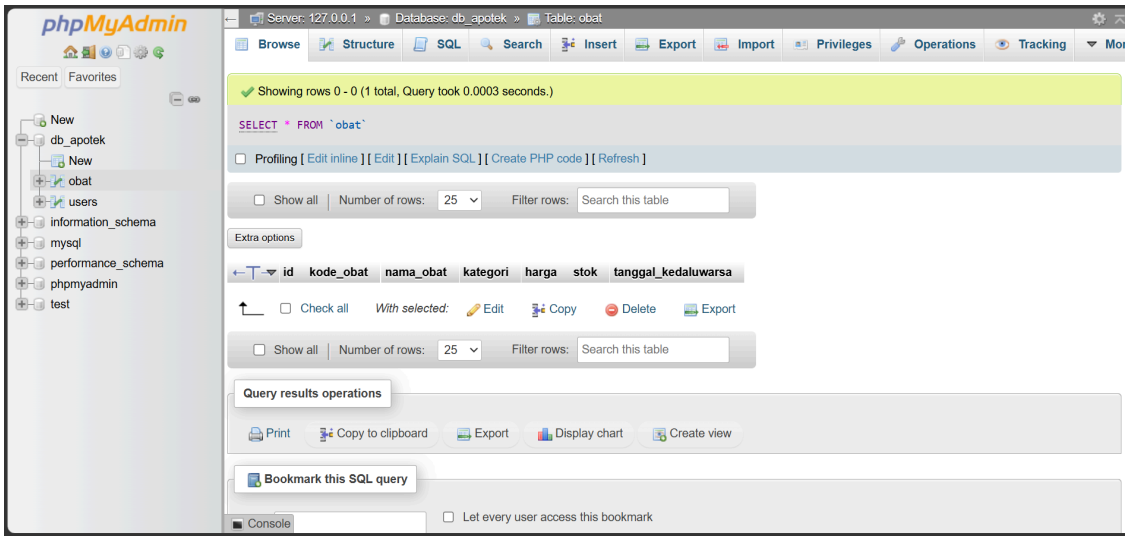
Tabel ini dipakai untuk sistem otentikasi (login). Kolomnya terdiri dari id *(Primary Key, Auto Increment*), username, password, dan nama_lengkap. Untuk alasan keamanan, data di kolom password tidak saya simpan dalam bentuk teks biasa (*plaintext*), melainkan menggunakan *hash* (enkripsi bcrypt) bawaan PHP.



Gambar Database Tabel User

B. Tabel obat

Tabel ini dipakai untuk proses CRUD moTdul apoteknya. Kolomnya berisi id, kode_obat, nama_obat, kategori, harga, stok, dan tanggal_kedaluwarsa. Kolom tanggal_kedaluwarsa ini yang nantinya akan jadi patokan saya untuk menerapkan algoritma pengurutan FEFO (*First Expired, First Out*).



Gambar Database Tabel Obat

2. Alur Integrasi & Koneksi Database

Setelah database siap, saya membuat file koneksi.php. File ini ibarat jembatan yang menghubungkan skrip PHP saya dengan database db_apotek. Sesuai dengan materi perkuliahan, saya menggunakan antarmuka PDO (*PHP Data*

Objects) yang lebih modern dibanding mysqli biasa, karena PDO sangat mendukung *Prepared Statements* untuk keamanan.

Alur pembuatannya dimulai dengan mendefinisikan variabel DSN (*Data Source Name*) yang memuat informasi *host* dan nama database. Kemudian, eksekusi koneksi (new PDO) saya letakkan di dalam blok *try-catch*. Kalau koneksinya berhasil, sistem jalan seperti biasa. Tapi kalau gagal (misalnya server MySQL mati), *error*-nya akan langsung ditangkap oleh catch dan program dihentikan dengan fungsi die(), jadi pesan *error* teknisnya tidak akan bocor berantakan di layar. File koneksi.php ini kemudian saya panggil di semua modul lain menggunakan *require_once*.

```php
<?php
// koneksi.php
$host     = "localhost";
$nama_db  = "db_apotek";
$username = "root"; // Sesuaikan dengan username database Anda (default XAMPP: root)
$password = "";     // Sesuaikan dengan password database Anda (default XAMPP: kosong)

// Data Source Name (DSN)[cite: 78]
$dsn = "mysql:host=" . $host . ";dbname=" . $nama_db . ";charset=utf8";

try {
    // Membuat instance PDO baru[cite: 78]
    $pdo = new PDO($dsn, $username, $password);

    // Set mode error PDO menjadi exception agar mudah ditangani[cite: 79]
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // Koneksi berhasil (tidak perlu di-echo agar tidak mengganggu tampilan HTML nantinya)
} catch (PDOException $e) {
    // Jika gagal, hentikan program dan tampilkan pesan error[cite: 80]
    die("Koneksi gagal: " . $e->getMessage());
}
?>
```

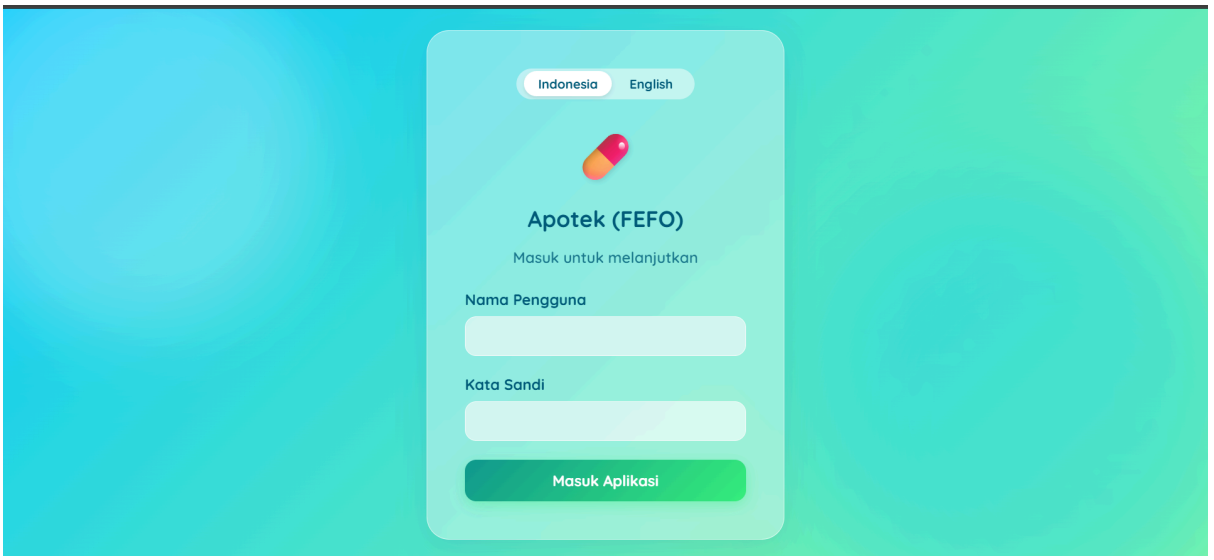
Gambar Koneksi Php

3. Halaman Login

Halaman ini saya buat untuk mengunci modul CRUD agar hanya admin yang punya akses. Saya menggunakan session_start() di awal skrip agar program bisa menggunakan variabel superglobal $_SESSION. Form loginnya memakai method="POST" supaya data *username* dan *password* yang diketik *user* tidak kelihatan di URL.

Alur logikanya: saat tombol login diklik, teks inputan disanitasi dulu pakai htmlspecialchars(), lalu dicek apakah kosong atau tidak menggunakan empty(). Kalau aman, PHP akan mencocokkan *username* ke tabel users. Karena *password* di database bentuknya *hash*, saya pakai fungsi password_verify() untuk mengecek kecocokannya. Kalau cocok, program akan membuat $_SESSION['user_id'] lalu mengarahkan *user* ke halaman Dasbor. Di halaman ini juga saya buat fitur ganti bahasa (Indonesia/English) yang preferensinya

disimpan ke dalam *session* agar bahasanya konsisten terus saat pindah-pindah halaman.



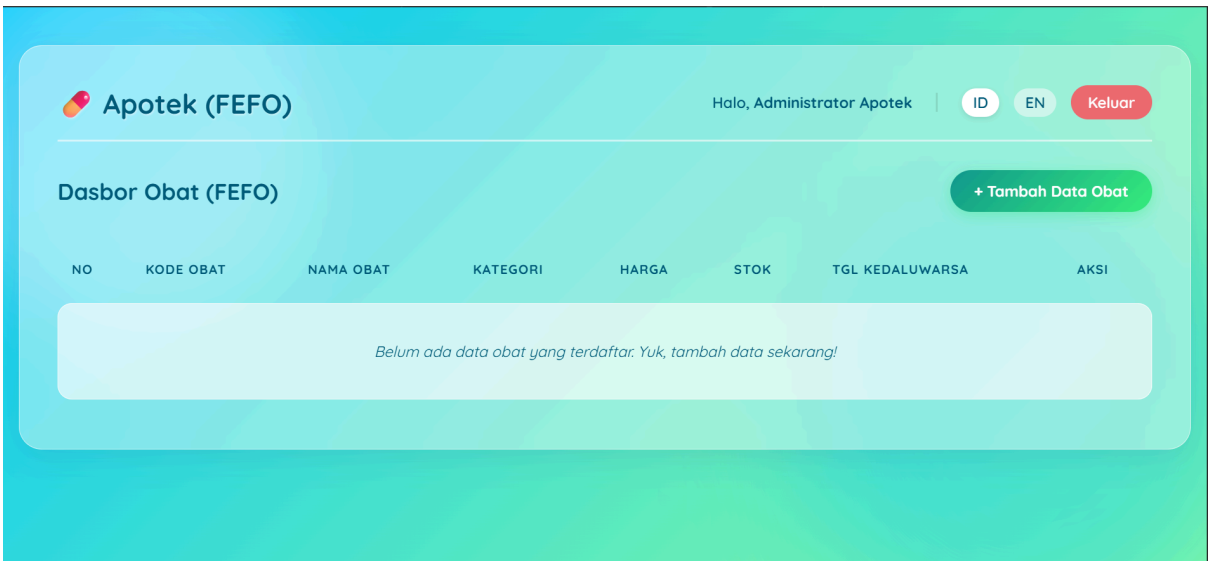
Gambar Halaman Login

4. Halaman Dasbor (Read)

Begitu berhasil login, *user* akan diarahkan ke Dasbor Utama. Halaman ini mewakili modul **Read** dari CRUD, yaitu membaca data dari database dan menampilkannya dalam bentuk tabel HTML.

Karena tema tugas ini Apotek, saya menerapkan konsep FEFO (*First Expired, First Out*). Caranya cukup *simple* tapi efektif, yaitu dengan memodifikasi *query* SQL-nya menjadi SELECT * FROM obat ORDER BY tanggal_kedaluwarsa ASC. Otomatis, obat yang masa kedaluwarsanya paling mepet akan ditarik ke urutan paling atas.

Untuk me-render datanya ke tabel, saya menggunakan perulangan foreach untuk mengurai *array* hasil *query* tersebut. Saya juga menambahkan logika validasi tanggal di PHP: kalau sisa waktu kedaluwarsa kurang dari 30 hari, tanggalnya akan otomatis ditandai dengan label/badge warna merah. Di atas tabel, ada tombol *Logout* yang berfungsi menjalankan perintah session_destroy() untuk membersihkan sesi login saat *user* keluar dari aplikasi.



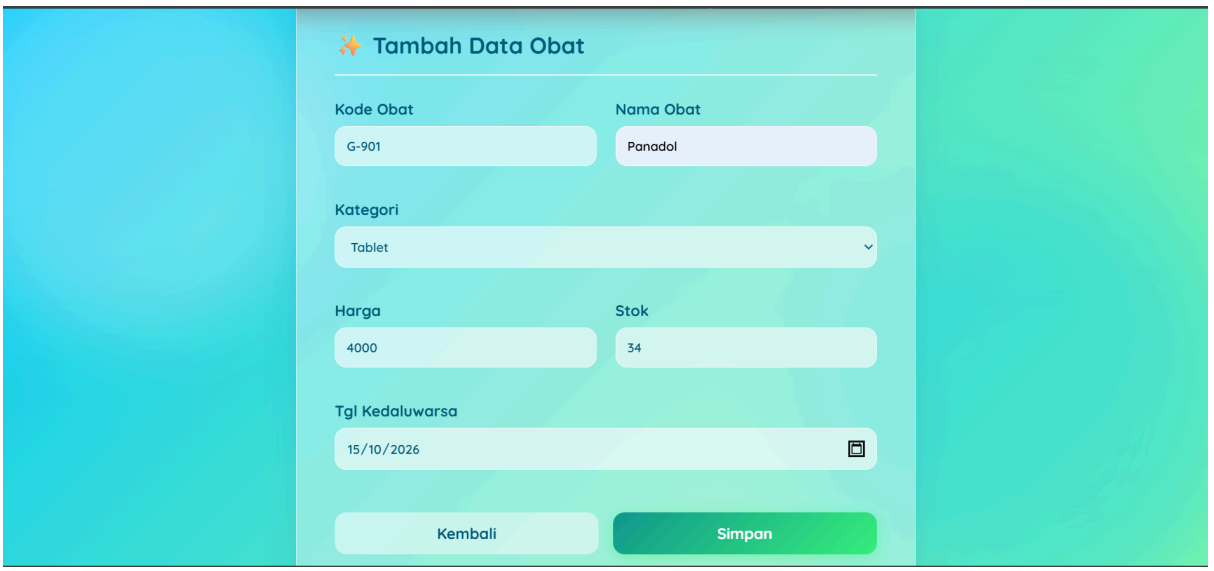
Gambar Halaman Dashboard

5. Halaman Tambah Data (Create & Validasi)

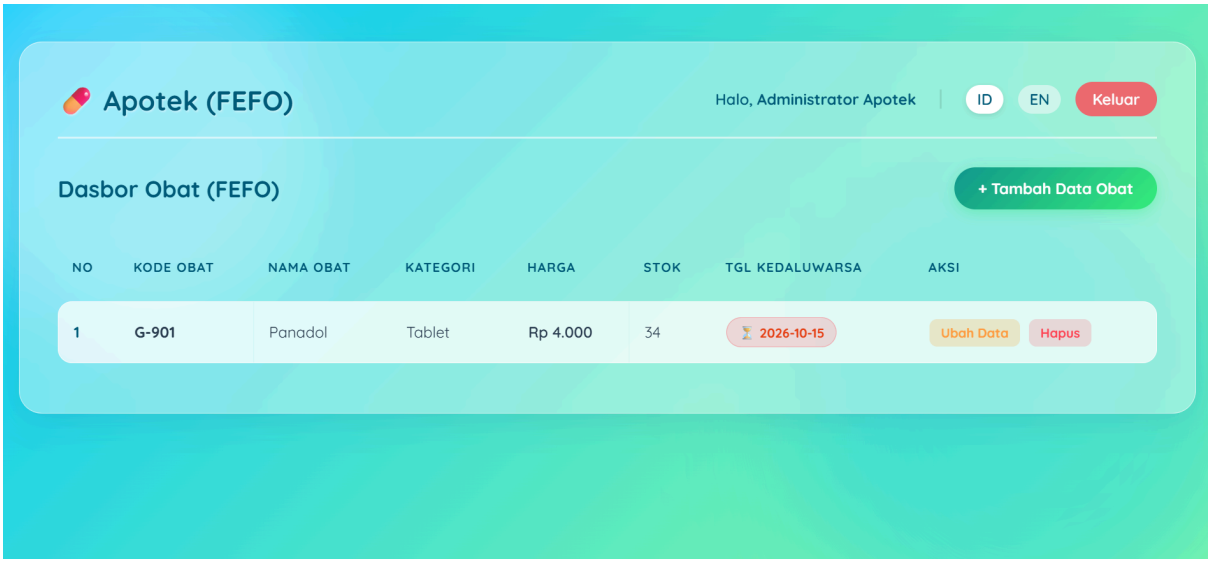
Form untuk modul **Create** (Tambah Data). Form ini saya atur agar melempar data menggunakan method="POST" ke file itu sendiri.

Alur penyimpanannya saya buat sangat ketat demi keamanan. Saat *user* klik Simpan, inputannya langsung diproses sanitasi. Saya pakai htmlspecialchars() untuk input teks (kode, nama, kategori), dan filter_var() untuk input angka (harga, stok) buat mencegah masuknya *script* berbahaya (XSS). Setelah itu, program mengecek kelengkapan data pakai if (empty(...)) supaya tidak ada data kosong yang masuk ke database.

Kalau lolos validasi, barulah data dilempar ke tabel lewat perintah INSERT INTO. Eksekusi *query* ini saya bungkus pakai fitur *Prepared Statements* dari PDO dengan parameter penanda (seperti :kode, :nama). Ini adalah langkah wajib untuk mencegah celah serangan *SQL Injection*. Kalau simpannya sukses, halaman otomatis me-*redirect* kembali ke Dasbor.



Gambar Halaman Tambah Data

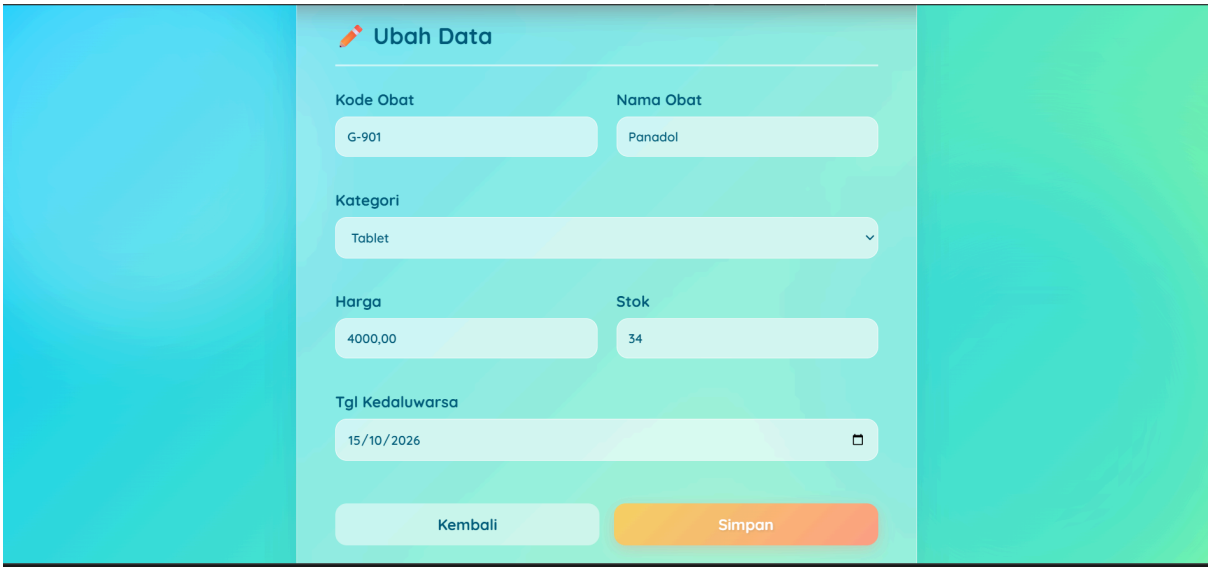


Gambar Halaman Dashboard yang berhasil menambahkan data

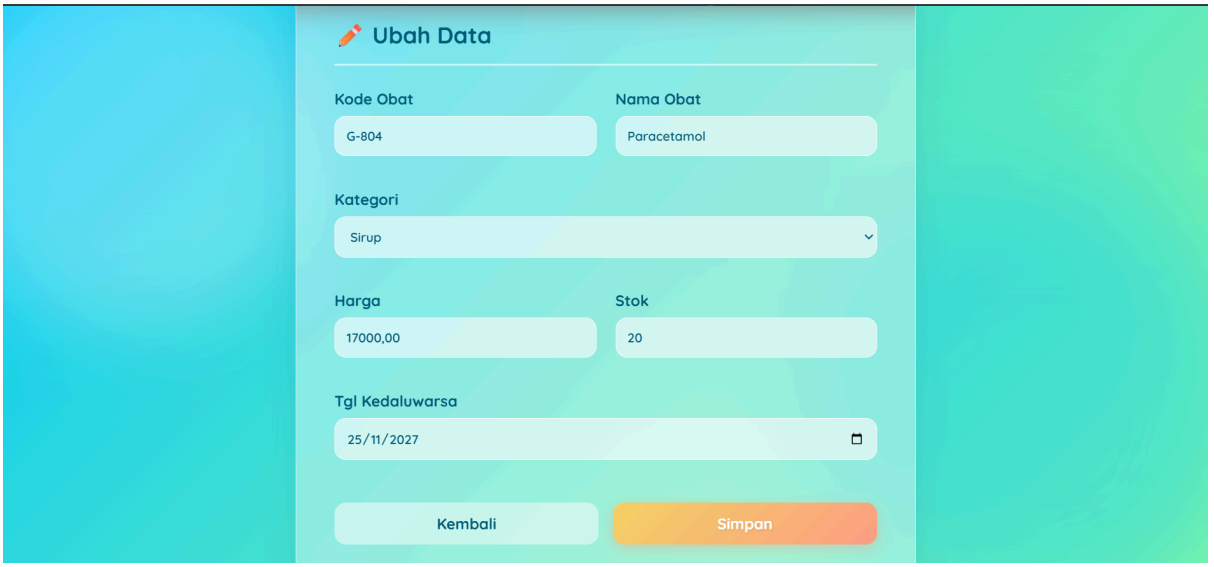
6. Halaman Ubah Data (Update)

Update (Ubah Data) ini secara tampilan mirip dengan form Tambah, tapi alur awalnya sedikit beda. Saat tombol "Edit" di baris tabel diklik, URL akan mengoper ID obat tersebut (contohnya edit.php?id=5). Skrip PHP akan menangkap parameter ID tersebut lewat $_GET['id'], lalu menjalankan *query* SELECT untuk menarik data lama obat itu dari database dan langsung mengisikannya ke dalam form (*pre-filled*).

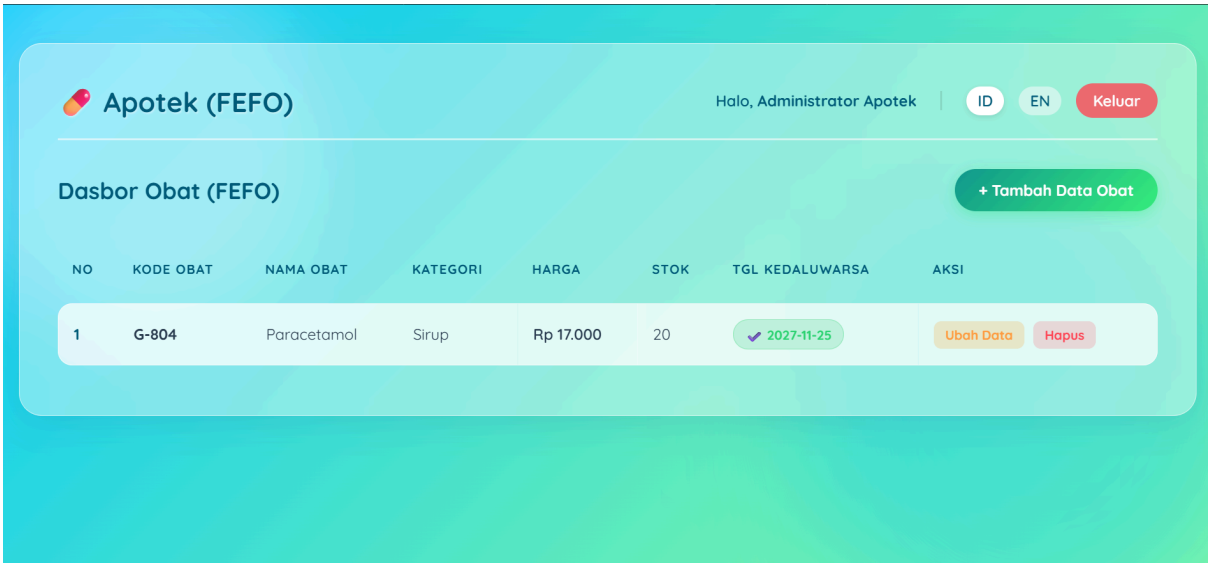
Kalau *user* mengubah nilainya dan klik simpan, proses sanitasi dan validasi data yang ketat akan tetap dijalankan seperti di form Tambah tadi. Bedanya, setelah lolos validasi, *query* yang dieksekusi adalah UPDATE obat SET ... WHERE id = :id. Eksekusinya juga sama-sama mengandalkan *Prepared Statements* PDO agar sistem tetap kebal dari *SQL Injection*.



Gambar Halaman Data Sebelum di Update



Gambar Halaman data yang akan di Update

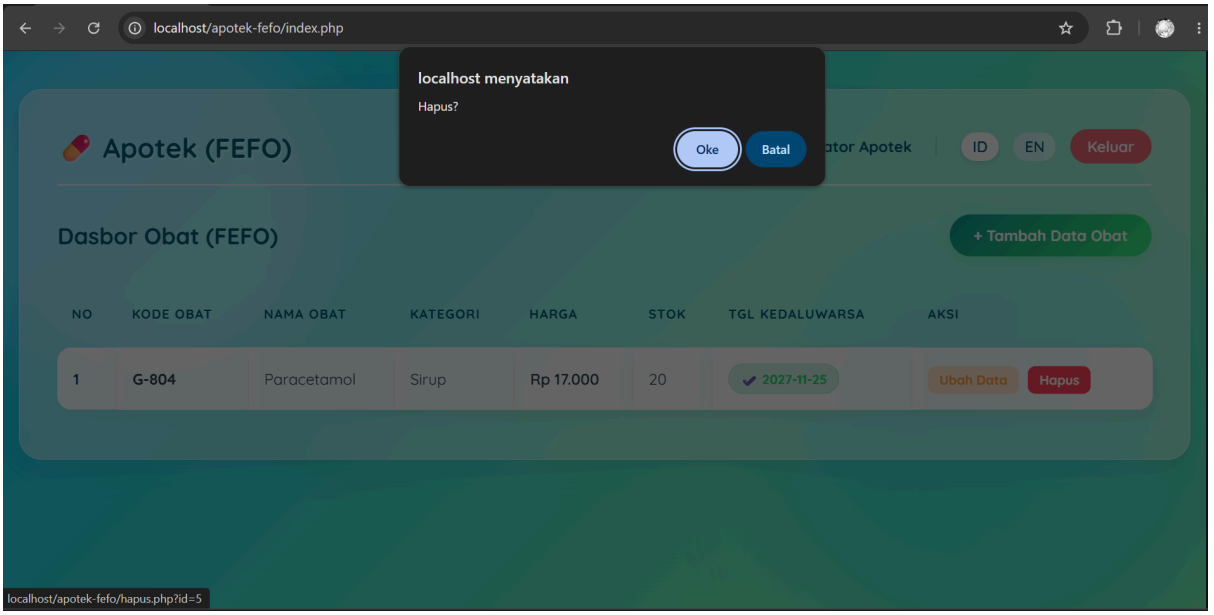


Gambar Halaman Dashboard yang datanya sudah di update tadi dihalaman update data

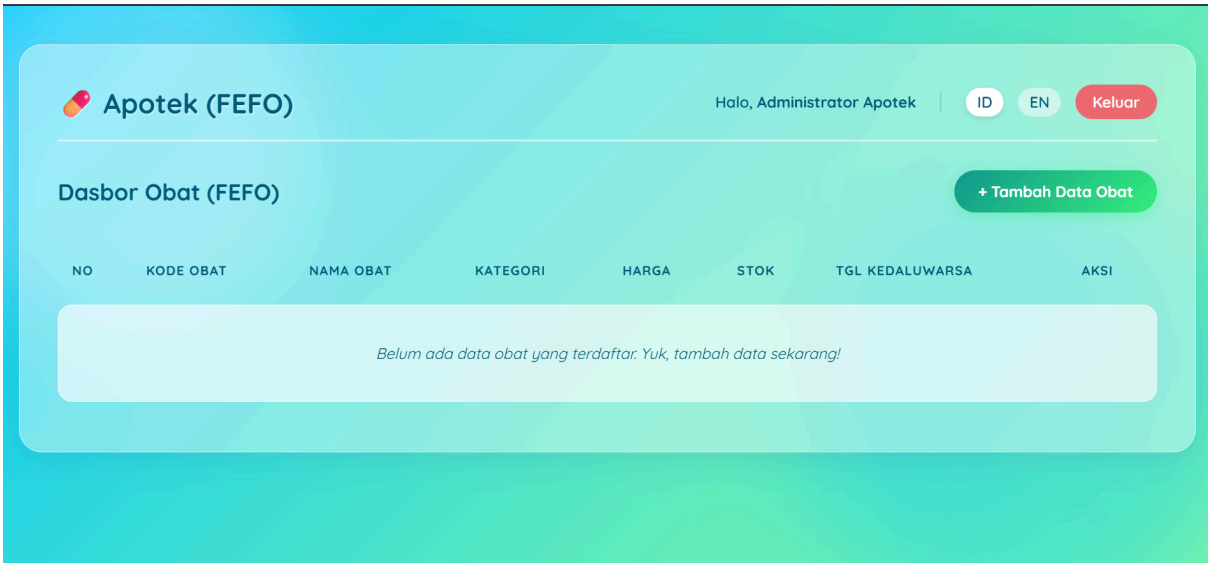
7. Fitur Hapus Data (Delete)

Untuk modul **Delete** (Hapus Data), saya sengaja tidak membuat halaman HTML atau form khusus supaya prosesnya lebih cepat (*seamless*). Alurnya: saat *user* menekan tombol "Hapus" pada salah satu baris obat di Dasbor, program akan memunculkan jendela *pop-up* konfirmasi dari *JavaScript* (return confirm). Ini berguna untuk mencegah data terhapus kalau *user* cuma tidak sengaja klik.

Kalau *user* memilih 'OK', barulah program pindah ke file hapus.php dengan membawa ID data (misal: hapus.php?id=3). Skrip PHP di dalamnya akan memvalidasi *session*, menangkap parameter ID, lalu langsung menjalankan *query* DELETE FROM obat WHERE id = :id menggunakan PDO *Prepared Statements*. Begitu data berhasil dihapus dari tabel, program secara otomatis menjalankan instruksi header("Location: index.php") untuk mengembalikan *user* ke halaman Dasbor seolah-olah tidak ada perpindahan halaman sama sekali.



Gambar Halaman Dashboard yang akan datanya dihapus



Gambar Halaman Dashboar yang Datanya Berhasil Dihapus